Table des matières

Cours 09 — Les codes 1
Étape 1 — La documentation sans trou · De bout en bout 1
Étape 2 — La commande qui rebâtit, et sa garde · De bout en bout 2
Étape 5 — Prouver : les lignes, puis les témoins · De bout en bout 3
Étape 6 — Répondre au mandat du café · MARTS 4
Étape 7 — Passer le projet · De bout en bout 5

Cours 09 — Les codes

Cours 9/9 · La Brûlerie — Formation Analytics Engineering

STAGING	INTERMEDIATE	MARTS
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
✓ ici · De bout en bout, le projet dbt complet, assemblé	✓	✓

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. Chaque fichier est rangé sous son chemin ; les descriptions et le `README.md` sont ceux du corrigé, et les vôtres peuvent le dire autrement. L'effacement, `effacer.sql`, n'est pas ici : la commande de l'étape 3.2 l'écrit.

Étape 1 — La documentation sans trou · De bout en bout

02_ae/models/staging/_staging.yml

```yaml
# 02_ae/models/staging/_staging.yml : les six vues du STAGING, chacune avec son grain et ce qu'elle fait à la RAW
models:
  # 1. la caisse : de chaque ticket et de chaque ligne, le dernier export, donc le 12 juin une seule fois
  - name: stg_tickets
    description: "Un ticket par ligne, typé, tiré de son export le plus récent."
  - name: stg_lignes
    description: "Une ligne de ticket par ligne, typée, tirée de son export le plus récent, avec l'instant de cet export."
  # 2. le catalogue et les membres : un seul export, typé et décodé
  - name: stg_produits
```


© 2026 iAlexMG. Tous droits réservés.

```
    description: "Un produit du catalogue par ligne, au 2025-06-30, son drapeau actif décodé en est_actif."
  - name: stg_membres
    description: "Un membre par ligne, sans rien qui permette de lui écrire : le secteur postal, l'inscription, les points,
l'infolettre."
  # 3. les deux journaux, typés : un lot, une décision par ligne
  - name: stg_lots
    description: "Un lot du torréfacteur par ligne, typé."
  - name: stg_consentements
    description: "Une décision d'un membre par ligne, typée."
```

02_ae/models/intermediate/_intermediate.yml

```
# 02_ae/models/intermediate/_intermediate.yml : la vue du milieu, avec la règle qu'elle porte
models:
  # 1. ce qui est une vente se décide ici, une fois, pour tout ce qui lit les ventes
  - name: int_ventes
    description: "Les lignes des tickets payés, avec leur date, leur membre et l'instant de leur export : les annulés sortent, le
remboursement reste."
```

02_ae/snapshots/_snapshots.yml

```
# 02_ae/snapshots/_snapshots.yml : l'histoire des prix, et ce qu'il faut savoir avant de l'effacer
snapshots:
  # 1. la seule table qui porte une histoire : elle se rebâtit parce que le journal du catalogue la garde
  - name: snp_prix
    description: "Une version par prix de chaque produit, tirée du journal du catalogue depuis 2022 ; elle se rebâtit regard par
regard, du plus ancien au plus récent."
```

Étape 2 — La commande qui rebâtit, et sa garde · De bout en bout

02_ae/macros/exiger_entrepot_neuf.sql

```
{# 02_ae/macros/exiger_entrepot_neuf.sql : la garde, qui refuse de rebâtir sur un entrepôt déjà bâti #}
{% macro exiger_entrepot_neuf() %}
    {# 1. ce que le projet a bâti, à ses préfixes ; Data Engineering a aussi ses tables ici #}
    {% set requete %}
        select count(*) from information_schema.tables
        where table_catalog = current_database()
            and regexp_matches(table_name, '^(stg|int|dim|fct|snp)_')
    {% endset %}
    {% set objets = run_query(requete).columns[0].values()[0] %}
```


© 2026 iAlexMG. Tous droits réservés.

```
    {# 2. un seul suffit : l'histoire des prix, rejouée par-dessus la sienne, s'abîmerait #}
    {% if objets > 0 %}
        {{ exceptions.raise_compiler_error(
            "L'entrepôt porte déjà " ~ objets ~ " objets du projet : rien n'a été touché."
        ) }}
    {% endif %}
    {{ log("Entrepôt neuf : il n'a que la RAW.", info=True) }}
{% endmacro %}
```

02_ae/reconstruire.cmd

```
@echo off
rem 02_ae/reconstruire.cmd : rebatir le projet de zero, en une commande, sur un entrepot sans aucun de ses objets
setlocal
rem 1. le projet est le dossier de ce script, ou que soit la copie ; l'entrepot est celui du profil
set DBT_PROJECT_DIR=%~dp0
rem 2. la garde d'abord : rien ne s'ecrit sur un entrepot qui porte deja le projet
dbt run-operation exiger_entrepot_neuf || exit /b 1
rem 3. l'histoire des prix, regard par regard, du plus ancien au plus recent
dbt snapshot --vars "{jour: 2022-12-31}" || exit /b 1
dbt snapshot --vars "{jour: 2023-12-31}" || exit /b 1
dbt snapshot --vars "{jour: 2024-12-31}" || exit /b 1
dbt snapshot --vars "{jour: 2025-06-30}" || exit /b 1
rem 4. tout le reste, tests compris, puis la documentation, qui lit ce qui est bati
dbt build || exit /b 1
dbt docs generate --static || exit /b 1
echo Reconstruction : au vert.
```

Étape 5 — Prouver : les lignes, puis les témoins · De bout en bout

02_ae/outils/comparer.py

```
# 02_ae/outils/comparer.py : l'entrepôt rebâti contre celui d'avant, objet par objet
import sys

import duckdb

# 1. les deux entrepôts, attachés côte à côte en lecture seule : celui d'avant, puis le rebâti
con = duckdb.connect()
con.execute(f"attach '{sys.argv[1]}' as avant (read_only)")
con.execute(f"attach '{sys.argv[2]}' as rebati (read_only)")
```


© 2026 iAlexMG. Tous droits réservés.

```python
# 2. les objets du projet, à leurs préfixes, avec leur type ; un objet d'un seul côté est un écart
objets = ("select table_name, table_type from information_schema.tables where table_catalog = ? "
          "and regexp_matches(table_name, '^(stg|int|dim|fct|snp)_')")
avant, rebati = (dict(con.execute(objets, [base]).fetchall()) for base in ("avant", "rebati"))
ecarts = len(avant.keys() ^ rebati.keys())
for nom in sorted(avant.keys() & rebati.keys()):
    if rebati[nom] == "VIEW":
        # 3. une vue, par sa définition, la même des deux côtés : elle nomme l'entrepôt qu'elle lit
        vue = "select count(distinct sql) - 1 from duckdb_views() where view_name = ?"
        ecart = con.execute(vue, [nom]).fetchone()[0]
        etat = "vue, définition " + ("différente" if ecart else "identique")
    else:
        # 4. une table, par ses lignes, dans les deux sens : ce qui manque, puis ce qui est en trop
        sens = "select count(*) from (select * from {} except all select * from {})"
        manquent = con.execute(sens.format(f"avant.{nom}", f"rebati.{nom}")).fetchone()[0]
        en_trop = con.execute(sens.format(f"rebati.{nom}", f"avant.{nom}")).fetchone()[0]
        ecart, etat = manquent + en_trop, f"table, {manquent} ligne(s) manquent, {en_trop} en trop"
    ecarts += ecart
    print(f"{nom:<18} {etat}")
print(f"{len(rebati)} objets rebâtis, {len(avant)} avant : {ecarts} écart(s)")
sys.exit(1 if ecarts else 0)
```

Étape 6 — Répondre au mandat du café · MARTS

6.5 Le café torréfié et le café vendu

```sql
-- 1. chaque fait additionné à son grain, puis les deux rangés côte à côte par le produit
with torrefie as (
    select cle_produit, count(*) as lots, sum(poids_torrefie_kg) as kg_torrefies
    from brulerie.fct_lots
    group by cle_produit
),
vendu as (
    select cle_produit, sum(quantite) as sacs
    from brulerie.fct_ventes
    group by cle_produit
)
select p.nom, t.lots, t.kg_torrefies, v.sacs, v.sacs * 0.340 as kg_vendus
from brulerie.dim_produit as p
inner join torrefie as t on t.cle_produit = p.cle_produit
```


© 2026 iAlexMG. Tous droits réservés.

```
left join vendu as v on v.cle_produit = p.cle_produit
order by t.kg_torrefies desc;
```

Étape 7 — Passer le projet · De bout en bout

02_ae/README.md

```
# 02_ae — l'étoile des ventes de La Brûlerie du Passage

<!-- 1. ce qu'est le dépôt, et ce qu'il livre -->
Le projet dbt d'Analytics Engineering de La Brûlerie, un café de quartier à Montréal. Il reprend la
`RAW` de juin 2025, la rend lisible, en tire l'étoile des ventes, le fait des lots du torréfacteur et
celui des consentements des membres, et prouve que les ventes retombent sur la caisse au cent près :
2 509 lignes, 16 718,00 $.

## Ce qu'il faut avoir

<!-- 2. ce qui n'est pas dans le dépôt : l'entrepôt, le profil, les outils -->
- `Projet_Brulerie/plateforme/entrepot/brulerie.duckdb`, avec les sept tables de la `RAW` ;
- `Projet_Brulerie/profiles.yml`, hors du dépôt : le profil `brulerie`, sa cible `dev` et le chemin
  `plateforme/entrepot/brulerie.duckdb` ;
- l'environnement conda `formations` : dbt-core 1.12.3, dbt-duckdb 1.11.0, sqlfluff 4.3.0, DuckDB.

## Rebâtir

<!-- 3. la commande, d'où la lancer, et ce qui doit sortir -->
Sur un entrepôt qui n'a aucun objet du projet, depuis `Projet_Brulerie`, dans l'Anaconda Prompt :
`02_ae\reconstruire.cmd`. Attendu : `PASS=43`, puis `Reconstruction : au vert.` Sur un entrepôt
qui porte déjà le projet, la garde refuse, et rien n'est touché.

## Prouver et vérifier

<!-- 4. les preuves, puis la vérification d'un changement -->
- `python 02_ae\outils\comparer.py <avant> plateforme\entrepot\brulerie.duckdb` : 0 écart contre un
  entrepôt mis de côté avant de rebâtir ;
- `dbt test --select test_type:singular` : le grain et les trois témoins, `PASS=4` ;
- `python 02_ae\ci\verifier.py` : le linteur, puis `dbt build` sur une copie neuve du dépôt et un
  entrepôt vide, `CI au vert`.
```


© 2026 iAlexMG. Tous droits réservés.